

高校图书馆智能管理系统 — 实验报告

技术栈: C++17 / Qt6 / SQLite / CMake / Ninja 编译环境: MinGW-W64 GCC 11.5.0 × Qt 6.8.2

一、系统概述

本系统是一个基于 **8 种核心数据结构** 构建的高校图书馆管理软件，涵盖图书管理、读者管理、借阅归还、图书推荐、热门排行、座位预约、消息队列、操作日志等完整业务功能。右侧提供 **数据结构可视化面板**、**消息队列面板** 和 **操作日志面板**（默认关闭，从视图菜单开启），直观展示每次操作对底层数据结构的影响。

数据结构与业务映射

数据结构	存储形态	应用场景
链表 (LinkedList)	单向链表	图书列表、读者列表、借阅记录
栈 (Stack)	链栈	操作撤销 (Undo)
队列 (Queue)	链队列	热门图书预约排队、消息队列
二叉排序树 (BST)	AVL 平衡树	按索书号索引图书
哈希表 (HashTable)	链地址法 + djb2	读者学号索引、ISBN/书名索引
图 (Graph)	邻接表	图书推荐网络（共同借阅关系）
最大堆 (MaxHeap)	完全二叉树数组	热门图书排行
稀疏矩阵 (SparseMatrix)	三元组压缩	座位分布管理

二、用户使用指南

2.1 启动与登录

运行 `LibrarySystem.exe`，弹出登录对话框：

- 输入 **学号/工号** 和 **密码** 进行身份验证
- 管理员账号： `admin` / `admin123` , 自动获得**全部权限**
- 读者账号： `2024001` / `123456` 等
- **权限由账号自动判定**，无需手动勾选管理员选项
- 新用户可点击「注册新读者」自助注册
- **关闭登录窗口则程序退出**，无法绕过登录

2.2 权限模型

功能	管理员	普通读者
图书管理	增删改查全部可见	仅搜索框 + 图书列表（只读）
读者管理 Tab	可见	完全隐藏
借阅/归还	可见	可见
图书推荐	可见	可见
热门排行	可见	可见
座位预约	可预约/释放任意座位	同一时段仅一个座位

2.3 功能标签页

登录后，左侧为 **6 个功能标签页**（普通读者为 5 个，读者管理隐藏）：

标签	图标	功能
图书管理		管理员：增删改查；读者：搜索 + 浏览
读者管理		仅管理员可见 — 增删改查读者
借阅/归还		借书、还书、撤销、库存不足时预约排队
图书推荐		输入 ISBN，基于借阅共现推荐相关图书
热门排行		展示借阅次数 Top N 的热门图书
座位预约		10×10 座位网格，自动使用登录身份

2.4 视图面板（从 "视图" 菜单开启，默认关闭）

- **数据结构可视化**：操作后自动展示 BST、哈希表、堆、图、栈、队列、稀疏矩阵的实时状态

- **消息队列**：展示所有写操作的 FIFO 处理历史，体现并发安全
- **操作日志**：记录所有重要操作，下拉框支持「全部 / 仅管理员」筛选

2.5 数据持久化

- **文件 → 保存修改**：将当前状态写入 SQLite 数据库 (`library.db`)
- **文件 → 放弃修改**：回滚到上次保存状态
- 关闭窗口时若有未保存修改会提示

三、系统架构

```
main.cpp
├─ MainWindow
│   └─ QTabWidget (左侧功能标签)
│       ├── BookTab      ↔ LibrarySystem ↔ 数据结构层
│       ├── ReaderTab    ↔ (仅管理员可见)
│       ├── BorrowTab    ↔ LibrarySystem
│       ├── RecommendTab ↔ LibrarySystem
│       ├── HotRankTab   ↔ LibrarySystem
│       └─ SeatTab       ↔ LibrarySystem
│
│   └─ QSplitter (右侧视图面板, 默认隐藏)
│       ├── VisualPanel    - BST/Hash/Heap/Graph/Stack/Queue/Matrix
│       ├── MessageQueueTab - 消息队列处理历史
│       └─ LogPanel        - 操作日志 (含筛选)
│
LibrarySystem (核心业务层)
├─ bookBST / readerHash / hotHeap / recommendationGraph / seatMatrix ...
├─ MessageQueueProcessor - 并发安全 (队列)
├─ OperationLogger       - 操作审计
└─ DataPersistence       - SQLite 持久化
```

四、功能模块实现细节

4.1 图书管理 — 链表 + BST + 哈希表

涉及数据结构： `LinkedList<Book*>`（主存储）、`BST<Book>`（索书号 AVL 索引）、`HashTable<Book>`（ISBN/书名索引）

4.1.1 添加图书 `LibrarySystem::addBook()`

```
Status LibrarySystem::addBook(Book* book)
{
    if (!book) return Status::Invalid;
    bookList.append(book);           // 链表 O(1)
    bookBST->insert(book);           // AVL 树 O(log n)
    bookTitleHash->insert(book);     // 哈希表 O(1)
    bookISBNHash->insert(book);      // 哈希表 O(1)
    hotHeap->insert(book);           // 堆 O(log n)
    recommendationGraph.addVertex(book->isbn); // 图 O(1)

    messageQueue->recordOperation(MsgType::AddBook, ...);
    operationLogger->log(currentOpId, currentOpAdmin, "添加图书", ...);
    return Status::OK;
}
```

BST 插入 (AVL 旋转) (`datastruct/BST.h`):

```
Node* insertNode(Node* node, const T& value) {
    if (!node) { nodeCount++; return new Node(value); }
    if (keyFn(value) < keyFn(node->data))
        node->left = insertNode(node->left, value);
    else if (keyFn(value) > keyFn(node->data))
        node->right = insertNode(node->right, value);
    // 平衡维护
    updateHeight(node);
    int balance = getBalance(node);
    if (balance > 1 && keyFn(value) < keyFn(node->left->data))
        return rotateRight(node); // LL
    if (balance < -1 && keyFn(value) > keyFn(node->right->data))
        return rotateLeft(node); // RR
    if (balance > 1 && keyFn(value) > keyFn(node->left->data)) {
        node->left = rotateLeft(node->left);
        return rotateRight(node); // LR
    }
    if (balance < -1 && keyFn(value) < keyFn(node->right->data)) {
        node->right = rotateRight(node->right);
        return rotateLeft(node); // RL
    }
}
```

```

    }
    return node;
}

```

哈希表插入 (djb2 + 链地址法) (datastruct/HashTable.h):

```

Status insert(const T& value) {
    if (static_cast<double>(count) / capacity > 0.75)
        rehash(capacity * 2);           // 负载因子 > 0.75 扩容
    size_t idx = hash(keyFn(value)) % capacity;
    Node* node = new Node(value);
    node->next = buckets[idx];           // 链地址法头插
    buckets[idx] = node;
    count++;
    return Status::OK;
}

size_t hash(const std::string& key) const {
    size_t h = 5381;                    // djb2 算法
    for (char c : key) h = ((h << 5) + h) + static_cast<unsigned char>(c);
    return h;
}

```

4.1.2 范围查询 (BST 中序遍历)

```

void rangeQueryNode(Node* node, const K& low, const K& high,
                    std::vector<T>& result) const {
    if (!node) return;
    K key = keyFn(node->data);
    if (key > low) rangeQueryNode(node->left, low, high, result);
    if (key >= low && key <= high) result.push_back(node->data);
    if (key < high) rangeQueryNode(node->right, low, high, result);
}

```

4.1.3 权限控制

```

// BookTab::setAdminMode()
void BookTab::setAdminMode(bool admin) {
    formBox->setVisible(admin); // 隐藏新增/编辑表单，仅保留搜索 + 列表
}

```

```
// MainWindow::onLoginSuccess()
funcTabs->setTabVisible(1, isAdmin); // 读者管理标签仅管理员可见
```

4.2 读者管理 — 链表 + 哈希表

涉及数据结构： `LinkedList<Reader*>` (主存储)、 `HashTable<Reader>` (学号索引)

4.2.1 登录验证 (无管理员勾选框)

```
// LoginDialog::onLogin()
bool valid = library->verifyLogin(id, pwd, adminLogin);
// adminLogin 直接从 verifyLogin 返回, 无需手动勾选
if (!valid) { /* 登录失败 */ return; }
accept();
```

```
// HashTable::find - O(1) 平均
T* find(const std::string& key) const {
    size_t idx = hash(key) % capacity;
    Node* cur = buckets[idx];
    while (cur) {
        if (keyFn(cur->data) == key) return &(cur->data);
        cur = cur->next; // 链地址遍历
    }
    return nullptr;
}
```

4.3 借阅/归还 — 栈 + 队列 + 图 + 堆

涉及数据结构： - `Stack<OperationRecord>` — Undo - `Queue<QString>` (多个) — 预约排队
- `Graph` — 推荐网络 - `MaxHeap<Book>` — 热门排行

4.3.1 借书操作

```
Status LibrarySystem::borrowBook(const QString& readerId, const QString& isbn)
{
    Reader* r = findReader(readerId); // 哈希 O(1)
    Book* b = findByISBN(isbn); // 哈希 O(1)
    if (!r || !b) return Status::NotFound;
    if (!r->canBorrow()) return Status::Full;
    if (b->availableStock <= 0) return Status::Full;
```

```

    b->availableStock--;
    b->borrowCount++;
    r->currentBorrow++;

    borrowRecords.append(new BorrowRecord(nextRecordId++, readerId, isbn));
    hotHeap->updateKey(b);                // 堆  $O(\log n)$ 

    // 图：共同借阅边 +1
    for (const auto& other : readerBooks)
        recommendationGraph.addEdge(b->isbn, other, 1);

    undoStack.push(op);                    // 栈  $O(1)$ 
    messageQueue->recordOperation(MsgType::BorrowBook, ...);
    operationLogger->log(currentOpId, currentOpAdmin, "借阅图书", ...);
    return Status::OK;
}

```

堆核心 (datastruct/Heap.h):

```

void siftUp(int i) {
    while (i > 0) {
        int p = (i - 1) / 2;
        if (keyFn(heap[p]) >= keyFn(heap[i])) break;
        std::swap(heap[p], heap[i]);
        i = p;
    }
}

void siftDown(int i) {
    int n = heap.size(), largest = i;
    int l = 2*i+1, r = 2*i+2;
    if (l < n && keyFn(heap[l]) > keyFn(heap[largest])) largest = l;
    if (r < n && keyFn(heap[r]) > keyFn(heap[largest])) largest = r;
    if (largest != i) { std::swap(heap[i], heap[largest]); siftDown(largest); }
}

```

4.3.2 还书 + 自动预约队列处理

```

Status LibrarySystem::returnBook(const QString& readerId, const QString& isbn)
{
    // ... 恢复库存、标记归还 ...
    // FIFO 出队下一个等待读者
}

```

```

auto it = reservationQueues.find(isbn);
while (it != reservationQueues.end() && !it->second.isEmpty()) {
    QString nextReaderId;
    if (it->second.dequeue(nextReaderId) != Status::OK) break;
    // 为下一位读者自动办理借书
}
undoStack.push(op);
messageQueue->recordOperation(MsgType::ReturnBook, ...);
return Status::OK;
}

```

队列核心 (datastruct/Queue.h):

```

Status enqueue(const T& value) {
    Node* node = new Node(value);
    if (rear) rear->next = node; else front = node;
    rear = node; ++length;
    return Status::OK;
}
Status dequeue(T& out) {
    if (!front) return Status::Empty;
    Node* temp = front; out = temp->data;
    front = front->next;
    if (!front) rear = nullptr;
    delete temp; --length;
    return Status::OK;
}

```

4.3.3 撤销操作

```

Status LibrarySystem::undoLastOperation()
{
    OperationRecord op;
    if (undoStack.pop(op) != Status::OK) return Status::Empty; // 栈弹出
    if (op.type == OperationType::BorrowBook) {
        // 撤销借书 = 还书 + 回滚图边权重
        b->availableStock++; r->currentBorrow--; b->borrowCount--;
    } else if (op.type == OperationType::ReturnBook) {
        // 撤销还书 = 重新借出
        b->availableStock--; r->currentBorrow++; b->borrowCount++;
    }
}

```

```
    return Status::OK;
}
```

栈核心 (datastruct/Stack.h):

```
Status push(const T& value) {
    Node* node = new Node(value);
    node->next = top; top = node; ++length;
    return Status::OK;
}
Status pop(T& out) {
    if (!top) return Status::Empty;
    Node* temp = top; out = temp->data;
    top = top->next; delete temp; --length;
    return Status::OK;
}
```

4.4 图书推荐 — 图

涉及数据结构: Graph (邻接表实现)

```
std::vector<std::pair<std::string, int>> Graph::recommend(
    const std::string& vertex, int topN) const
{
    auto neighbors = getNeighbors(vertex);
    std::sort(neighbors.begin(), neighbors.end(),
        [](auto& a, auto& b) { return a.second > b.second; });
    // 取前 topN
}
```

```
class Graph {
    std::map<std::string, std::vector<std::pair<std::string, int>>> adj;
public:
    void addEdge(const std::string& from, const std::string& to, int weight) {
        if (from == to) return;
        auto& neighbors = adj[from];
        for (auto& [nb, w] : neighbors)
            if (nb == to) { w += weight; return; }
        neighbors.push_back({to, weight});
        // 对称更新 (无向图)
    }
};
```

```
    }  
};
```

4.5 热门排行 — 最大堆

```
std::vector<Book*> LibrarySystem::getHotBooks(int topN) const {  
    return hotHeap->getTopN(topN);  
}  
// MaxHeap::getTopN - Floyd 建堆 + 依次取堆顶
```

4.6 座位预约 — 稀疏矩阵

涉及数据结构： `SparseMatrix<Seat>`（三元组压缩）。

座位仅两种状态： `Free` / `Occupied`，空闲座位不占存储。

```
template <typename T>  
class SparseMatrix {  
    struct Triplet { int row, col; T value; };  
    std::vector<Triplet> data;  
public:  
    void set(int r, int c, const T& val) {  
        for (auto& t : data)  
            if (t.row == r && t.col == c) { t.value = val; return; }  
        data.push_back({r, c, val});  
    }  
    T* getPtr(int r, int c) {  
        for (auto& t : data)  
            if (t.row == r && t.col == c) return &t.value;  
        return nullptr;  
    }  
    std::vector<std::vector<T>> toDenseMatrix(int rows, int cols, T def) const {  
        std::vector<std::vector<T>> result(rows, std::vector<T>(cols, def));  
        for (const auto& t : data) result[t.row][t.col] = t.value;  
        return result;  
    }  
};
```

座位预约逻辑（自动使用登录学号，读者单座位限制）：

```

void SeatTab::onReserve()
{
    // 自动使用 currentUserId, 无需手动输入学号
    // 普通读者检查 isReaderSeatConflict() → 已有座位则拒绝
    if (!isAdmin && isReaderSeatConflict(currentUserId)) {
        QMessageBox::warning("预约限制", "同一时段只能预约一个位置");
        return;
    }
    // 检查目标座位空闲
    Seat* s = library->getSeat(r, c);
    if (s && !s->isFree()) { /* 已被占用 */ return; }
    library->reserveSeat(r, c, currentUserId, start, end);
}

void SeatTab::onRelease()
{
    if (!isAdmin) {
        // 自动扫描网格找到读者唯一座位并释放
        auto grid = library->getSeatGrid(10, 10);
        for (int r = 0; r < 10; ++r)
            for (int c = 0; c < 10; ++c)
                if (grid[r][c].readerId == currentUserId) {
                    library->releaseSeat(r, c);
                    return;
                }
        QMessageBox::information("你没有预约任何座位");
        return;
    }
    // 管理员：释放选中座位
    library->releaseSeat(rowSpin->value(), colSpin->value());
}

```

网格着色： ● 绿色=空闲 ● 红色=他人占用 ● 橙色=自己的座位

4.7 消息队列 — 多读者并发安全

涉及数据结构： `Queue<OperationMessage*>`（链队列，FIFO）

所有写操作自动经过消息队列序列化。核心方法 `recordOperation()` 在 `LibrarySystem` 各写操作成功后调用，创建预处理消息入队并立即处理记录到历史。

```

void MessageQueueProcessor::recordOperation(
    MsgType type, const QString& readerId, bool succeeded,
    const QString& resultText, ...)
{
    auto* msg = new OperationMessage(...);
    msg->processed = true;          // 预完成, 跳过重复执行
    enqueue(msg);
    processAll();                  // 记录到历史 (最近 500 条)
}

```

4.8 操作日志 — 用户审计

涉及组件: `OperationLogger` (`std::vector<LogEntry>` 存储 + Qt 信号)

```

struct LogEntry {
    QDateTime timestamp;
    QString operatorId;    // 操作人学号
    bool isAdmin;          // 是否管理员
    QString operation;     // "添加图书" / "借阅图书" / "预约座位" ...
    QString details;       // 详情描述
};

```

9 种写操作均记录日志, `LogPanel` 通过 `QComboBox` 筛选「全部 / 仅管理员」。

4.9 数据持久化 — SQLite

```

bool DataPersistence::saveAll(LibrarySystem* sys) {
    db.transaction();
    sys->getBookList().traverse([&](Book* b) { /* INSERT */ });
    sys->getReaderList().traverse([&](Reader* r) { /* INSERT */ });
    // ... borrow_records, seats, reservations
    db.commit();
}

// 启动时 loadAll() → 数据库为空时 SeedData::populate() 填充 30 本书 + 5 读者

```

五、数据结构可视化面板

`VisualPanel` 通过 `QGraphicsScene` 渲染:

- **BST**：中序遍历，显示节点总数、树高度
- **哈希表**：遍历桶，展示键值映射、容量与负载
- **最大堆**：数组形式，堆顶标注
- **图**：顶点数、边数、邻接关系
- **栈**：栈顶到栈底的操作历史
- **队列**：各预约队列长度与等待读者
- **稀疏矩阵**：10×10 座位网格（绿/红/橙三色）

六、关键业务校验

校验点	实现
登录验证	哈希表 O(1) + 密码比对
读者存在性	<code>readerHash->find(id)</code>
借阅上限	<code>r->canBorrow()</code>
库存检查	<code>b->availableStock > 0</code>
座位冲突	<code>getPtr(r,c) → isFree()</code>
读者单座位	<code>isReaderSeatConflict()</code> 扫描网格
权限控制	<code>setAdminMode()</code> + <code>setTabVisible()</code>
并发安全	消息队列 FIFO 序列化

七、总结

本系统以高校图书馆为场景，实现了 **8 种数据结构** 的工程应用：

1. **链表** — 图书/读者/借阅记录主容器
2. **AVL 树** — 索书号高效索引与范围查询
3. **哈希表** (djb2 + 链地址) — O(1) 读者/图书查找
4. **栈** — 操作撤销
5. **队列** — 预约排队 + 消息队列
6. **图** (邻接表) — 共同借阅推荐

7. **最大堆** — 热门图书排行

8. **稀疏矩阵** — 座位分布管理

配合 **Qt6** 图形界面、**SQLite** 持久化、**消息队列** 并发模型、**操作日志** 审计、**权限分级** (管理员/读者) , 完整演示了数据结构在实际软件系统中的作用与价值。